

RABT: Run-Ahead Block TAGE

Abstract—CBP-NG’s VFS metric for branch predictors makes latency, throughput, and energy first-class design constraints alongside accuracy. This paper presents Run-Ahead Block TAGE (RABT), a block-level TAGE predictor that issues ahead-of-use table reads and predicts up to seven conditional branches in a 256-instruction fetch block. To make run-ahead prediction robust to successor ambiguity, RABT uses a compact secondary tag derived from the successor block’s PC. RABT further combines independent per-branch resolution chains with energy-conscious banking, allocation, and metadata updates. On the CBP-NG suite, RABT achieves a VFS of 0.897, with an MPKI of 8.96, IPC of 9.01, CPI of 0.081, and EPI of 2837 fJ, while reducing prediction latency to under one cycle. These results show that a latency-oriented block predictor can deliver high throughput and energy efficiency while maintaining competitive accuracy.

I. INTRODUCTION

CBP-NG’s VFS-based scoring evaluates branch predictors by accuracy, latency, throughput, and energy. This shifts the design objective away from maximizing accuracy alone and toward predictors that can deliver low-latency, high-throughput predictions within a tight energy budget.

Prior CBP predictors, especially TAGE-SC-L variants, achieve high accuracy with large storage budgets and long resolution chains [1]. Under VFS-based scoring, increased predictor complexity is penalized with increased latency and energy. Figure 1 shows that, across a range of MPKI operating points, the VFS gain from a one-cycle latency reduction consistently exceeds the marginal value of further accuracy improvement, motivating a latency-first design. Run-ahead prediction is attractive in this setting because it moves table accesses off the timing-critical prediction path and eliminates short mispredictions by avoiding multi-layer predictor disagreements.

This paper presents Run-Ahead Block TAGE (RABT), a predictor that combines TAGE [3] with the run-ahead prediction approach of Cai et al. [2]. RABT extends run-ahead TAGE prediction to fetch-block granularity, predicting up to $N=7$ conditional branches in a 256-instruction block at sub-cycle latency. Our contributions are:

- We extend run-ahead TAGE prediction to support predicting up to seven conditional branches per block.
- We utilize dynamic counters to track rolling accuracy and allocation pressure to assist in dynamic training/allocation policy.
- We combine independent per-branch resolution chains with VFS-oriented banking and floorplan-aware RAM ordering to reduce latency and energy.
- We utilize a number of techniques from literature to improve accuracy while operating under tight energy and layout constraints.

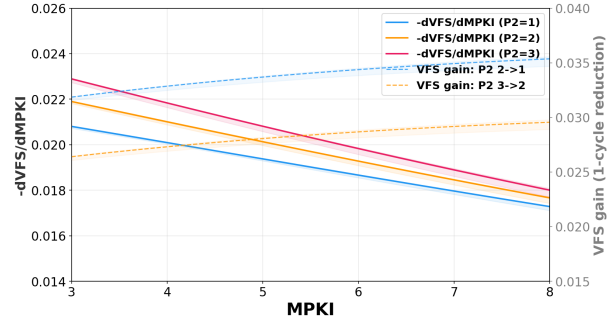


Fig. 1. Marginal VFS Value: Accuracy vs. Latency Reduction

II. DESIGN OVERVIEW

A. Ahead Pipeline

Figure 2 shows the two-stage ahead pipeline used by RABT. The predictor operates on fetch blocks containing up to $N=7$ conditional branches within a 256-instruction region. Unlike prior run-ahead TAGE work [2], which runs a fixed number of instructions ahead, RABT runs one fetch block ahead and produces an N -wide prediction vector for the next block.

The key detail is that table reads for a block are issued before the frontend demands predictions for that block (deemed a *successor block*). When block B is being predicted, RABT has already performed SRAM reads for the relevant TAGE entries during the prediction generation for the previous block.

As such, the P1 stage just needs to compute secondary tag hits and resolve providers for each branch, enabling subcycle prediction latency. The primary tag match hit computation is performed in the previous stage of the pipeline, splitting the long resolution chain into two shorter ones across the pipeline stage. While issuing predictions for B , RABT also performs SRAM reads for the next successor block in parallel. Figure 3 shows the temporal overlap of prefetch, predict, and train across cycles, including the two-phase training on misprediction.

a) Prefetch and resolve (P1): P1 performs two operations. First, it disambiguates the pipeline register entries using a secondary tag computed with the current block’s PC. It then runs N independent resolution chains, matching entries to branches by AND-ing secondary and primary tag hits (including the 3 upper bits of the primary tag to indicate branch number/slot in the block), to select providers, alternates, and apply meta corrections for the current block. Secondly, it issues SRAM reads for the successor block so that its entries are available in the next cycle, and precomputes tag matches using only the 8-bit htag (the 3 slot bits are ignored till resolution which happens in the next call after the pipeline

shift), latching results in pipeline registers, after they have been consumed for the current cycle's predictions.

b) Predict stage (P2): P2 returns the already-latched prediction bit computed in P1 for the current block. As such, P2 has the same sub-cycle latency as P1 and always returns the same prediction, eliminating short mispredictions and associated energy losses due to frontend flushes.

c) Training: Training operates on state captured in pipeline registers during prediction, decoupling update latency from prediction latency. On every resolved block, the provider's hysteresis counter is updated (increment on correct, decrement on mispredict) and the usefulness counter is incremented when the provider was correct and the alternate disagreed. The meta counter is updated when the provider was weak and provider/alt disagreed. On misprediction, allocation searches tables above the provider for an entry with $u=0$. A single entry is allocated with the actual outcome and zeroed counters; when no candidate exists, usefulness counters above the provider are decremented. Mispredicted blocks require an additional cycle: the fallback hysteresis must be read before the fallback update can proceed, and allocation must write to the single-ported tag and secondary tag RAMs without conflicting with the ahead reads.

B. Table Configuration

Table I (appendix) summarizes the final RABT configuration. The total RAM budget is 75 KB; Table II (appendix) gives the full breakdown. History lengths follow a geometric series across 15 tables. Table sizes use a two-tier scheme: 1024 entries for the longest-history tables (T0–T4) and 2048 entries for the remaining tables (T5–T14).

a) Entry-point indexing: Table indices are computed by XORing folded global history with $PC \gg 2$ (word-aligned), making them unique to the *entry point* of a fetch block. Tags use $PC \gg 5$ (block-aligned), so the tag identifies the block while the index distinguishes entry points within it. This decouples block identity from entry point, allowing the predictor to maintain distinct histories for different paths into the same block.

b) Floorplan-aware RAM ordering: The physical ordering of RAM declarations affects wire delay (latency) and energy. Each TAGE table's RAMs are organized into two clusters separated by a zone boundary. The *predict cluster* (tag, fold, sec) contains the RAMs read on the timing-critical path during P1; placing folded-history registers immediately after the tag RAM minimizes the wire delay for tag comparison, which is the tightest constraint for meeting sub-cycle latency. The *update cluster* (pred, hyst, u) contains the remaining RAMs. Figure 4 shows the physical layout.

C. Independent Resolution Chains

With $N=7$ branches and a modest number of tables, branch slots may compete for provider and allocation entries, performing poorly on high branch-density blocks. We explored block-level provider selection by packing all N predictions

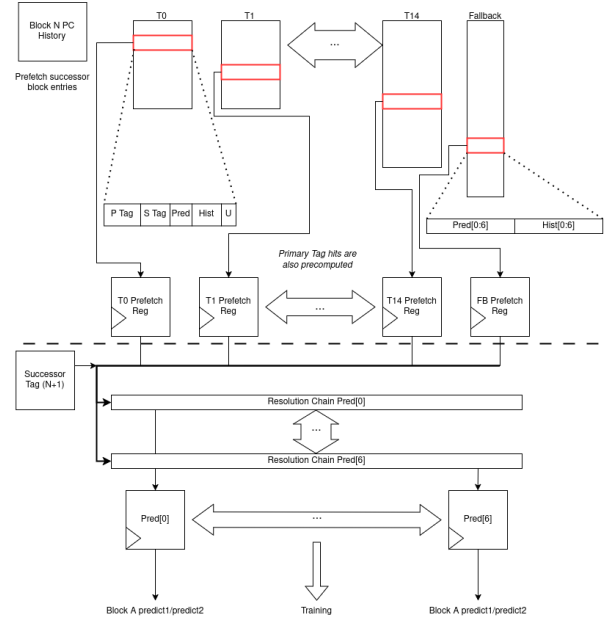


Fig. 2. Two-Stage Ahead Pipeline

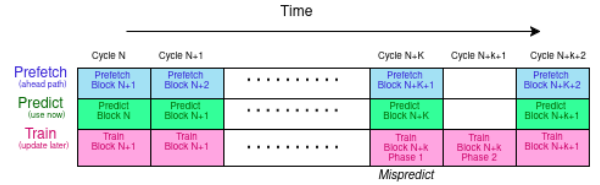


Fig. 3. Pipeline Operation Across Cycles

into a single entry, but this forces every branch in the block to use the same history depth and is suboptimal.

Instead, each branch runs its own independent resolution chain with separate provider, alternate, and meta arbitration. A 3-bit group identifier distinguishing the branch within the block is encoded in the tag field (Figure 5).

The effective comparison tag (htag) is $11 - 3 = 8$ bits. This trades intra-block correlation for per-branch independence, yielding a 2.0% MPKI improvement and 7.2% EPI reduction compared to block-level selection. Workloads with strong intra-block correlation regress; we analyze this in the appendix.

III. DETAILED OPERATION

A. Secondary Tag

A block with N branches has up to $N+1$ successor blocks (one taken target per branch, plus the fall-through). The ahead pipeline prefetches TAGE entries for the successor to the current block. Each entry stores a 5-bit secondary tag computed from the PC of the block that allocated it. During P1 resolution, the current block's PC is hashed with the same 3-way XOR function (Figure 5) and compared against the stored value to verify that the prefetched entries belong to the actual successor block requiring predictions. Mismatches are



Fig. 4. Physical RAM Layout

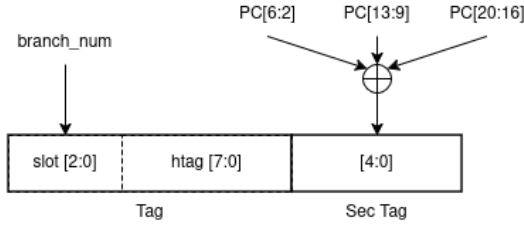


Fig. 5. Tag(s) format

rejected, falling through to shorter-history tables or bimodal. We use block PC rather than missing history as the basis for the secondary tag. See the appendix for the full sweep of widths and hash functions.

B. Banked RWRAM

Our ahead design issues reads every cycle for the next block, while training writes arrive from resolved blocks, potentially targeting the same bank. Each table uses 8 banks, which helps avoid read-write conflicts between prediction and training. It also reduces total SRAM area activated per write, aiding power optimization.

In case of a conflict, the write is buffered in a register and attempted on the next cycle if there is no conflict. If a new write arrives before the buffer flushes, the old write is lost.

8-bank uniform indexing was optimal; fewer banks increase write loss, more increase latency (see appendix).

C. Pressure Counters

Two saturating 10-bit counters track predictor health. The *accuracy counter* increments on correct predictions and decrements on mispredictions. The *allocation counter* increments

when no allocation slot is found and decrements when allocation succeeds. We tested a number of dynamic policies driven by these counters (gated decay, adaptive thresholds, secondary tag enforcement). Only allocation skip proved beneficial, described in the next subsection.

D. Allocation and Decay

On a misprediction, one entry is allocated above the provider (probabilistically allocating more than one led to pollution, -3.3% VFS). During allocation, entries whose htag matches but secondary tag or group ID differs are skipped, promoting allocation to the next table; this avoids evicting useful entries from other successor paths or branch groups [2]. When allocation pressure is high, the closest candidate table is probabilistically skipped, pushing allocation toward longer-history tables. When no table has a useless entry ($u=0$), allocation is suppressed entirely. Usefulness is managed via LFSR-gated probabilistic decay [7]. On each primary tag or secondary tag miss, if an 8-bit LFSR value falls below a threshold, the matched entry’s u -bit is decremented (3.1% chance of decay). Entries being allocated in the same cycle are exempt.

Epoch-based bulk reset, graded LFSR widths, and per-table thresholds were tested but uniform probabilistic decay proved more stable (see appendix).

E. Fallback Predictor

The fallback is a direct-mapped bimodal table with 8192 entries, indexed by $PC \gg 2$ (outperforming $PC \gg 5$ by disambiguating entrypoints within a block). It participates as the lowest-priority “table” in provider selection (hit bit hardwired to 1). A separate half-sized RAM stores per-branch hysteresis with a “flip only when wrong twice” policy, reducing fallback MPI by 9.1% and improving VFS by 1.9%.

F. Meta Corrections

Following [3], we use a pipelined meta corrector to apply bias corrections to alternate/provider selection [4]. When the provider is weak and the provider and alternate disagree, a 4-bit signed counter (2048 entries, pipeline depth 2) determines whether to trust the provider or fall back to the alternate.

G. Other Optimizations

All SRAM writes include a value-change check, eliminating redundant writes for power efficiency. Prediction counters flip only when hysteresis is weak and the prediction was wrong.

IV. METHODOLOGY

Development follows a two-step process: (1) a heavily-parameterized HARCOM behavioral model with a μ architectural monitor, used for rapid parameter sweeps, and (2) a hand-coded HARCOM model with explicit floorplanning for accurate area and timing. We iterate between the two, using gradient ascent on VFS to converge on the final configuration. Evaluation used three tiers: a quick evaluation (20 traces), full evaluation (all 168 CBP-NG traces), and μ Arch monitor evaluations (deep single-trace instrumentation for diagnostics). See the appendix for details.

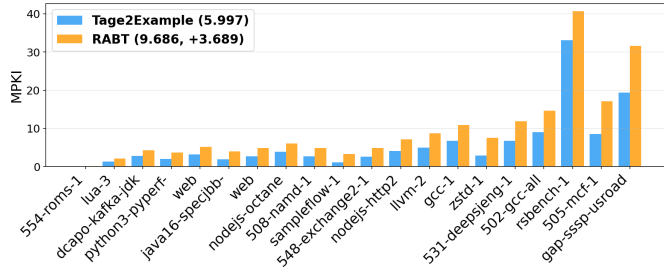


Fig. 6. Per-Trace MPKI: 2-Cycle TAGE vs. RABT

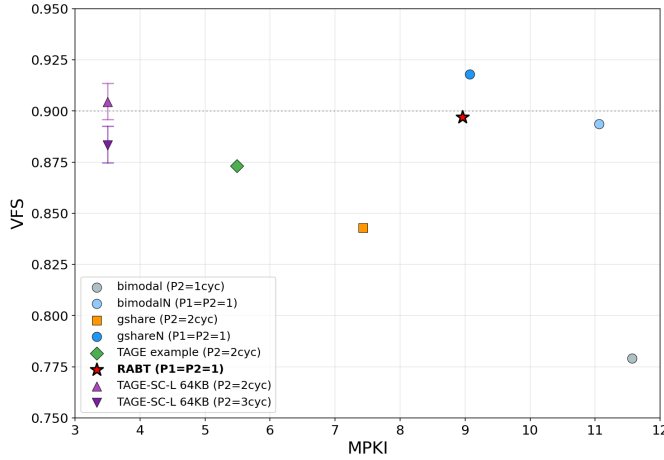


Fig. 7. VFS vs. MPKI for Different Predictors

V. RESULTS

Figure 6 compares the accuracy of RABT against the example 2-cycle TAGE provided with the CBP-NG framework on the 20 traces used for *quick evaluation*. RABT incurs an average MPKI increase of 3.7 compared to the 2-cycle baseline. For comparison, Cai et al. [2] report only +0.1 MPKI for their ahead predictor, but their design predicts individual branches ahead (not blocks), uses 21 TAGE tables at 56.6 KB, and a prediction queue. Additionally, the difference in fetch block granularity between the example predictor (16 instructions) and our ahead predictor (256 instructions) also contributes to the accuracy degradation.

A test shrinking the size of our Fetch Block with ahead prediction only showed a 1.1% decrease in MPKI. Examining the reason for MPKI degradation requires greater investigation.

Despite the accuracy penalty, the VFS gain from latency reduction (2 cycles $\rightarrow$ 1 cycle) more than compensates under the CBP-NG scoring framework.

Table III (appendix) decomposes the VFS improvement incrementally through the design space exploration for RABT.

Additional per-trace results, design space exploration (banking, history lengths, tag widths), and parameter sweep data are in the appendix.

VI. DISCUSSION

a) *VFS landscape*: Figure 7 plots VFS against MPKI for the predictors shipped with the CBP-NG framework (bimodal, gshare, TAGE, and their banked variants) alongside RABT. We also model a 64 KB TAGE-SC-L using the CBP-6 reference MPKI of 3.50 and the IPC/CPI measured from the example TAGE predictor. Because TAGE-SC-L has not been implemented in HARCOM, its EPI is estimated from the example TAGE EPI (1246 fJ) plus the overhead of 6–8 additional SC table reads, giving a range of 1800–3000 fJ shown as whiskers. We plot two latency scenarios: P2=2 cycles (optimistic; assumes SC is pipelined away or omitted) and P2=3 cycles (SC index computation depends on the TAGE provider output, requiring an extra cycle). Even under the optimistic assumptions, TAGE-SC-L’s $2.5\times$ MPKI advantage over RABT translates to only a modest VFS lead, which vanishes entirely at P2=3. This illustrates the diminishing returns of accuracy once latency and energy are accounted for.

b) *Run-ahead overrides*: RABT does not include a statistical correlator (SC), loop predictor (L), or IMLI component [6]. In conventional TAGE-SC-L, the SC overrides the TAGE prediction based on the provider and alternate values. Porting this to a run-ahead architecture is non-trivial: the SC inputs are not available until the resolution chain completes, which happens in the same cycle the prediction is needed. A BTB tracking second-degree successors could speculatively generate SC overrides ahead of time, serving a role analogous to meta corrections but with greater accuracy.

c) *Future work*: With latency at one cycle, the next gains lie in reducing MPKI and energy. Statistical correlator architectures adapted for ahead operation could recover much of the accuracy gap to TAGE-SC-L. BTB integration with shared entries could extend the ahead block distance, and a prediction dispatch queue are promising avenues.

VII. CONCLUSION

We have presented Run-Ahead Block TAGE (RABT), a branch predictor that adapts ahead pipelining to block-level TAGE with independent resolution chains, secondary tag disambiguation, and floorplan-aware RAM ordering. On the CBP-NG suite, RABT achieves a VFS of 0.897, with MPKI of 8.96, IPC of 9.01, and EPI of 2837 fJ. By incorporating the feedback discussed in Section VI and iterating on the ahead architecture, we VFS scoring sets up a more realistic evaluation framework for branch prediction. Under this framework, run-ahead architectures can perform as well as, if not better than, the conventional TAGE-SC-L variants that have dominated CBP championships so far.

REFERENCES

- [1] A. Seznec, “TAGE-SC for CBP2025,” presented at CBP, Tokyo, Japan, Jun. 2025.
- [2] L. Cai, A. Deshmukh, and Y. Patt, “Enabling ahead prediction with practical energy constraints,” in *Proc. ISCA*, 2025, doi: 10.1145/3695053.3730998.

- [3] A. Seznec, “A new case for the TAGE branch predictor,” in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Porto Alegre, Brazil, 2011, pp. 117–127.
- [4] A. Seznec, “A 256 Kbits L-TAGE branch predictor,” *J. Instruction-Level Parallelism*, vol. 9, 2007. [Online]. Available: <http://www.jilp.org/vol9>
- [5] A. Seznec, S. Felix, V. Krishnan, and Y. Sazeides, “Design tradeoffs for the Alpha EV8 conditional branch predictor,” in *Proc. 29th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2002, pp. 295–306.
- [6] A. Seznec, J. San Miguel, and J. Albericio, “The inner most loop iteration counter: A new dimension in branch history,” in *Proc. 48th Int. Symp. Microarchit. (MICRO)*, 2015, pp. 347–357.
- [7] A. Perais and A. Seznec, “Cost effective speculation with the omnipredictor,” INRIA, Rennes, France, Tech. Rep., 2015.

APPENDIX

A. Configuration and Results Tables

TABLE I
TAGE TABLE PARAMETERS

Parameter	Value
Number of tables (NT)	15
History range	$H_{\min}=8$ to $H_{\max}=200$ (geometric)
History lengths	{200, 158, 126, 100, 79, 63, 50, 40, 31, 25, 20, 15, 12, 10, 8}
Path history width	27 bits
Path bits per cycle	6 bits of next-PC
History buffer	8192 entries
Table sizes	1024 (T0–T4), 2048 (T5–T14)
Tag width	Uniform 11 bits (all tables)
Prediction counter	1-bit (CTR_WIDTH=1)
Hysteresis	2-bit (HYST_WIDTH=2, separate [5])
Usefulness counter	2-bit (U_WIDTH=2)
Secondary tag	5-bit (hash of successor PC)
Banks per table	8 (BANK_SHIFT=1)

TABLE II
RAM CAPACITY BREAKDOWN (75 KB TOTAL)

RAM type	Bits	KB	%
Tag (11b)	281,600	34.38	45.8
Sec tag (5b)	128,000	15.62	20.8
Fallback (7b)	57,344	7.00	9.3
Usefulness (2b)	51,200	6.25	8.3
Fallback hyst (7b)	28,672	3.50	4.7
Prediction (1b)	25,600	3.12	4.2
Hysteresis (2b)	25,600	3.12	4.2
Meta (4b)	16,384	2.00	2.7
<i>By table group (%)</i>			
T0–T4 (1024×5)		12.50	
T5–T14 (2048×10)		50.00	
FB + FB_BH		10.50	
Meta		2.00	

TABLE III
CUMULATIVE ABLATION

Design point	Δ VFS	MPKI	P2 (ns)	EPI (fJ)
2-cycle TAGE (example)	—	5.49	2 cyc	1246
RABT (shared resolution)	+0.9%	9.15	0.890	2571
+ Independent chains	+1.3%	9.09	0.867	2386
+ Fallback hysteresis	+2.7%	8.96	0.913	2837

B. Design Space Exploration

C. Write Loss and Bank Conflicts

The μ Arch monitor revealed 30–64% write loss on short-history tables (T13–T14) with 8-way banking; long-history tables showed near-ideal bank distribution. Per-bit flip rate instrumentation shows that short-history fold registers shift by only 1 bit per cycle, so consecutive indices differ by very few bits. At most 1 of the 3 bank-select bits flips between consecutive accesses, producing 56–57% same-bank rates on these tables.

TABLE IV
PARAMETER SWEEP SUMMARY

Parameter	Final	Alternatives tested
NT	15	14, 16 (timing fail)
$H_{\max}$	200	150, 300
Tag	Uniform 11b	Graded $\langle 13, 9 \rangle$, $\langle 11, 7 \rangle$
SEC_TAG	5-bit hash	3b, 4b, adaptive, press-gated
MAX_ALLOC	1	2 (-3.3% VFS)
FB capacity	8192	16384 (VFS drop from EPI)
META	4-bit/2048	2-bit/1024
Banks	8	2, 4 (write loss), 16 (EPI penalty)
BANK_SHIFT	Uniform 1	Graded $5 \rightarrow 1$ ($+0.27$ MPKI)
PC_IDX_SHIFT	2	5 (loses PC entropy, $+0.8$ MPKI)
FB banking	Single $8K \times 7$	4 addr banks ($+21\%$ P2)
FB hysteresis	4096×7	None (-1.9% VFS)
Decay	LFSR=8, thresh=8	Graded LFSR/thresh (all worse)

TABLE V
SECONDARY TAG SWEEP

Bits	Hash ($a \oplus b \oplus c$)	Rej.%	MPKI Δ
0	—	0%	+1.8%
3	PC _{4:2} , PC _{9:7} , PC _{14:12}	22.9%	+3.5%
4	PC _{5:2} , PC _{11:8} , PC _{16:13}	23.4%	+2.4%
5	PC _{6:2} , PC _{13:9} , PC _{20:16}	24%	baseline

We attempted several mitigations. Counter-based bank rotation increased write loss to 78% and MPKI by 97%, because rotating the bank bit changes which physical entry is accessed. XOR-ing block PC into bank select failed because the PC is constant within a self-succeeding loop. Per table bank indexing ($+0.27$ MPKI, 5/20 wins) and uniform bank indexing with offsets $=5$ ($+0.8$ MPKI) were both strictly worse. In hot self-looping blocks, read-write conflicts on short-history tables cannot be mitigated regardless of bank selection scheme. However, if the loop continues without mispredictions, the skipped training writes are benign since no update is needed. As such, this effort was concluded.

D. Workload Sensitivity to entry sharing

Independent resolution trades intra-block correlation for per-branch independence. This works well on average (-2.0% MPKI) but regresses on workloads with strong intra-block correlation (e.g. dcapo-kafka $+63.9\%$, sampleflow $+35.9\%$).